

Lecture 1

Static Type Checking (in Python)

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Motivating Example

```
def last(seq):  
    if len(seq) == 0:  
        return None  
    elif len(seq) == 1:  
        return seq[0]  
    else:  
        return last(seq[1:])  
  
lists = [[3, 1], [4], [1, 5, 9], [2]]  
sum_of_last = sum(last(L) for L in lists)
```

Does this code look fine to you?

Motivating Example

```
def last(seq):  
    if len(seq) == 0:  
        return None  
    elif len(seq) == 1:  
        return seq[0]  
    else:  
        return last(seq[1:])  
  
lists = [[3, 1, 4], [], [1, 5, 9], [2]]  
sum_of_last = sum(last(L) for L in lists)
```

How about now?

Type Errors

```
Traceback (most recent call last):  
  File "/Users/daryll/Downloads/cry.py", line 10, in <module>  
    sum_of_last = sum(last(L) for L in lists)  
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

- You can't add an int and None together.
- Could we have found this bug sooner?
- If we add **type hints**, we could!

Type Hints

- A **type hint** is an *explicit* declaration of types of variables, arguments, and return values.

```
num = 67
names = ["Koyuki", "Yuuka", "Hoshino"]
affirmations = (True, 1)
counter = {"pish": 1, "chicken": 2}
```

```
num: int = 67
names: list[str] = ["Koyuki", "Yuuka", "Hoshino"]
affirmations: tuple[bool, int] = (True, 1)
counter: dict[str, int] = {"pish": 1, "chicken": 2}
```

Type Hints

variables

```
<variable>: <type> = <value>
```

```
num: int = 67
```

tuples (fixed-size)

```
<variable>: tuple[<type-1>, ..., <type-N>] = <value>
```

```
affirmations: tuple[bool, int] = (True, 1)
```

Type Hints

tuples (variable size)

```
<variable>: tuple[<type>, ...] = <value>
```

```
nums: tuple[int, ...] = (3, 1, 4, 1, 5) # (3, 1, 4) also OK
```

lists/sets (elements must have *same* type)

```
<variable>: list[<type>] = <value>
```

```
<variable>: set[<type>] = <value>
```

```
names: list[str] = ["Koyuki", "Yuuka", "Hoshino"]
```


Type Hints

dicts (keys must have *same* type; values must have *same* type)

```
<variable>: dict[<key-type>, <value-type>] = <value>  
counter: dict[str, int] = {"pish": 1, "chicken": 2}
```

functions

```
def <function-name>(<arg>: <arg-type>, ...) -> <return-type>:  
def fib(n: int) -> int:
```

Why bother?

Suppose you want to modify the following code:

```
def catch(pokemon):  
    ...
```

- What is the expected type of `pokemon`?
- What type does `catch` return?

We need to go through the function body to find out.

Why bother?

How about now?

```
def catch(pokemon: dict[str, int]) -> bool:  
    ...
```

- What is the expected type of `pokemon`?
- What type does `catch` return?

No need to look through the function body anymore!

Why bother?

Why not...

```
def catch(pokemon):  
    """  
    Here, pokemon is a dict where the keys are strings and the  
    values are integers.  
  
    This function returns a bool.  
    """  
    ...
```

What do type hints provide that comments cannot?

Type Checking

- **Type checking:** process of checking whether type rules of a programming language are violated by code
- **Type error:** violation of a type rule
 - e.g., *you can't add an `int` with a `str`*

```
def f(a, b):  
    return a + b
```

```
f(1, 2)      # OK
```

```
f(3, "4")    # NG
```

Kinds of type checking

- Static type checking
 - done **before** code execution
 - Python **does not** do this by default
- Dynamic type checking
 - done **during** code execution
 - Python **does** do this by default

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

If we add type hints...

```
def f(a: int, b: int) -> int:  
    return a + b
```

```
f(1, 2)    # OK
```

```
f(3, "4")  # NG
```

Argument of type "Literal['4']" cannot be assigned to parameter "b" of type "int" in function "f"

"Literal['4']" is not assignable to "int"

Note that we *didn't* have to run the program!

Static type checkers

- A *static type checker* checks your code for type errors *before* you run it
- Recommended static type checkers for CS 12:
 - **Pyright**: made by Microsoft ([docs](#), [GitHub repo](#))
 - called Pylance if you use VS Code : (
 - much more mature (if you'll choose one, choose Pyright)
 - **ty**: made by Astral  ([docs](#), [GitHub repo](#))
 - same company that made `ruff` and `uv`
 - very young (beta release was just last month)

Pyright strict mode

- For CS 12, all of your (Python) submissions will be assessed under Pyright's *strict mode*
 - All function *parameters* **must** be annotated

```
def add(x: int, y: int): # no need to annotate return value
    ...
```

- Variables initialized as *empty* collections **must** be annotated

```
nums: list[int] = [] # OK
nums = []           # NG
```

Type Inference

What is the return type of this function?

```
def classify(n: int):  
    if n > 0:  
        return "POSITIVE"  
    elif n < 0:  
        return "NEGATIVE"
```

- `n > 0`: `str` ("POSITIVE")
- `n < 0`: `str` ("NEGATIVE")
- `n == 0`: `None`

Type Inference

You can also use static type checkers to find out!

```
def classify(n: int) -> (Literal['POSITIVE', 'NEGATIVE'] | None)
```

- `|`: Union type
 - `A | B` means *A or B*
- `Literal[<val-1>, ..., <val-N>]`: Literal type
 - `Literal[1, 2, 3]` means *1 or 2 or 3*

Limitations

How about this?

```
def classify(n: int):  
    if n > 0:  
        return "POSITIVE"  
    elif n < 0:  
        return "NEGATIVE"  
    elif n == 0:  
        return "EGG"
```

```
... -> (Literal['POSITIVE', 'NEGATIVE', 'EGG'] | None)
```

Limitations

How about *this*?

```
def classify(n: int):  
    if n > 0:  
        return "POSITIVE"  
    elif n < 0:  
        return "NEGATIVE"  
    else:  
        return "EGG"
```

```
... -> Literal['POSITIVE', 'NEGATIVE', 'EGG'] # there we go!
```

Type Narrowing

Pyright can *rule out possibilities* of union types using *conditionals*:

```
def f(nums: list[int] | None) -> int | None:
    if nums is None: # (1)
        return None

    nums.append(12) # (2)

    return sum(nums)
```

The `if` block removes the `None` possibility!

Type Alias

- **Type alias:** alternative name for *existing* type
 - Makes a type *equivalent* to the existing type
 - Simplifies writing complex types
 - Allows for *domain-specific* type names

```
def add_matrices(a: list[list[int]], b: list[list[int]]) -> list[list[int]]:  
    ...
```

```
type Matrix = list[list[int]] # remove `type` if using Python 3.12 or older  
  
def add_matrices(a: Matrix, b: Matrix) -> Matrix:  
    ...
```

Back to motivating example

```
def last(seq):  
    if len(seq) == 0:  
        return None  
    elif len(seq) == 1:  
        return seq[0]  
    else:  
        return last(seq[1:])  
  
lists = [[3, 1, 4], [], [1, 5, 9], [2]]  
sum_of_last = sum(last(L) for L in lists)
```

You can use type hints & Pylint to catch this!

Exhaustiveness checking

```
from typing import Literal

type Attack = Literal["Normal", "Explosive", "Piercing", "Mystic", "Sonic"]
type Armor = Literal["Normal", "Light", "Heavy", "Special", "Elastic"]
type Synergy = Literal["Resist", "Normal", "Weak", "Effective"]
```

For the next slide:

What literal type(s) will be inferred for each call to `reveal_type`?

- `reveal_type` is a *debug stub* provided by Pyright

Exhaustiveness checking

```
def synergy(attacker: Attack, defender: Armor) -> Synergy:
    # (note that this is very wrong)
    if attacker == "Normal":
        reveal_type(attacker); reveal_type(defender)
        return "Normal"
    elif attacker == "Explosive":
        return "Weak"
    elif attacker == "Piercing":
        if defender == "Normal" or defender == "Special":
            return "Normal"
        reveal_type(defender)
        return "Weak"
    reveal_type(attacker)
```